

DAY 3 — TABLE EXTRACTION (DUAL MODE FINAL)

WHERE WE ARE

Parser  (dual mode)

Titles 

Text 

Tables  (not usable yet)

TODAY GOAL

FREE → extract tables from text 

PAID → extract tables using GCP 

FILE STRUCTURE

app/

```
|—— parser/
|   |—— table_parser.py       FREE
|   |—— gcp_table_parser.py   PAID
|   |—— table_router.py       SWITCH
```

STEP 1 — FREE TABLE LOGIC

 app/parser/table_parser.py

```
import re
```

```
def is_table_line(line):
```

```
    nums = re.findall(r"\d+.\d*", line)
```

```
    return len(nums) >= 2
```

```
def extract_raw_tables(text_lines):
```

```
    tables = []
```

```
    current = []
```

```
    for line in text_lines:
```

```
        line = line.strip()
```

```
        if is_table_line(line):
```

```
            current.append(line)
```

```
        else:
```

```
            if len(current) >= 3:
```

```
                tables.append(current)
```

```
            current = []
```

```
    if len(current) >= 3:
```

```
        tables.append(current)
```

```
    return tables
```

```
def split_row(line):
```

```
    parts = re.split(r"\s{2,}", line)
```

```
    if len(parts) <= 1:
```

```
        parts = line.split()
```

```
    return [p.strip() for p in parts if p.strip()]
```

```
def structure_table(lines):
```

```
rows = [split_row(l) for l in lines]
```

```
max_len = max(len(r) for r in rows)
```

```
norm = []
```

```
for r in rows:
```

```
    if len(r) < max_len:
```

```
        r += [""] * (max_len - len(r))
```

```
    norm.append(r)
```

```
return {
```

```
    "columns": norm[0],
```

```
    "rows": norm[1:]
```

```
}
```

```
def build_tables(text_lines):
```

```
    raw_tables = extract_raw_tables(text_lines)
```

```
    result = []
```

```
    for t in raw_tables:
```

```
        structured = structure_table(t)
```

```
        if len(structured["rows"]) >= 2:
```

```
            result.append(structured)
```

```
    return result
```

✅✅ STEP 2 — PAID (GCP DOCUMENT AI)

 `app/parser/gcp_table_parser.py`

```
from google.cloud import documentai
```

```
def gcp_table_extract(file_path):
```

```
    # Replace these with your values
```

```
    project_id = "YOUR_PROJECT_ID"
```

```
    location = "us"
```

```
    processor_id = "YOUR_PROCESSOR_ID"
```

```
    client = documentai.DocumentProcessorServiceClient()
```

```
    name = f"projects/{project_id}/locations/{location}/processors/{processor_id}"
```

```
    with open(file_path, "rb") as f:
```

```
        file_content = f.read()
```

```
    request = documentai.ProcessRequest(
```

```
        name=name,
```

```
        raw_document=documentai.RawDocument(
```

```
            content=file_content,
```

```
            mime_type="application/pdf"
```

```
        )
```

```
    )
```

```
    result = client.process_document(request=request)
```

```
    doc = result.document
```

```

tables = []

for page in doc.pages:
    for table in page.tables:
        rows = []
        for row in table.body_rows:
            r = []
            for cell in row.cells:
                r.append(cell.layout.text_anchor.content.strip())
            rows.append(r)

        if rows:
            tables.append({
                "columns": rows[0],
                "rows": rows[1:]
            })

return tables

```

..

STEP 3 — TABLE ROUTER

 `app/parser/table_router.py`

```

from app.config import MODE

from app.parser.table_parser import build_tables

from app.parser.gcp_table_parser import gcp_table_extract

```

```
def get_tables(file_path, parsed_data):  
    if MODE == "paid":  
        return gcp_table_extract(file_path)  
  
    return build_tables(parsed_data["text"])
```

✅ STEP 4 — PIPELINE UPDATE

 `app/ingestion/pipeline.py`

```
from app.parser.parser_router import parse_document  
from app.parser.table_router import get_tables
```

```
def process_pdf(file_path):  
    parsed = parse_document(file_path)  
  
    tables = get_tables(file_path, parsed)  
  
    return {  
        "titles": parsed["titles"],  
        "text": parsed["text"],  
        "tables": tables  
    }  
``
```

✅ STEP 5 — TEST

```
from app.ingestion.pipeline import process_pdf  
  
result = process_pdf("data/sample.pdf")
```

```
print("\n=== TABLES ===\n")
```

```
for t in result["tables"]:
```

```
    print(t)
```

```
    print("-----")
```

✓ ✓ HOW TO USE

✓ FREE MODE

MODE = "free"

👉 uses:

Unstructured + your table logic

✓ PAID MODE

MODE = "paid"

👉 requires:

GOOGLE_APPLICATION_CREDENTIALS=path_to_json

👉 uses:

Document AI tables

✓ ✓ EXPECTED OUTPUT

✓ FREE MODE

✓ tables detected

✗ messy alignment (fixed Day 4)

✓ PAID MODE

✓ clean tables

✓ correct columns

✓ accurate values

✓ ✓ FINAL STATUS

Dual mode ✓

Tables ✓ (both implemented)

Switch ✓ working

✓ ✓ FINAL ANSWER

👉 This version:

✓ Includes FREE ✓

✓ Includes PAID ✓

✓ Same pipeline ✓

✓ One toggle ✓

✓ Next → Day 4 (dual mode cleaning — critical step) 🚀